НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Навчально-науковий інститут атомної та теплової енергетики
Кафедра цифрових технологій в енергетиці

ЗВІТ

з практичної роботи «Kafka»
з дисципліни «Проектування систем з розподіленими базами даних в енергетиці»

Виконав:
студент групи ТР-62мп
Заклецький Вадим

Перевірив:
Асистент Головакін М.А.

Київ – 2026

Тема: Основи роботи з Apache Kafka у розподілених базах даних.

Мета:

– ознайомитися з архітектурою Apache Kafka та її роллю в обробці потокових даних;

– навчитися створювати топіки, продюсерів і конс’юмерів;

– відпрацювати передачу повідомлень у розподіленому середовищі.

Скріншоти результатів роботи

Крок 1–2. Підготовка робочого середовища, завантаження та встановлення Kafka 3.7.1

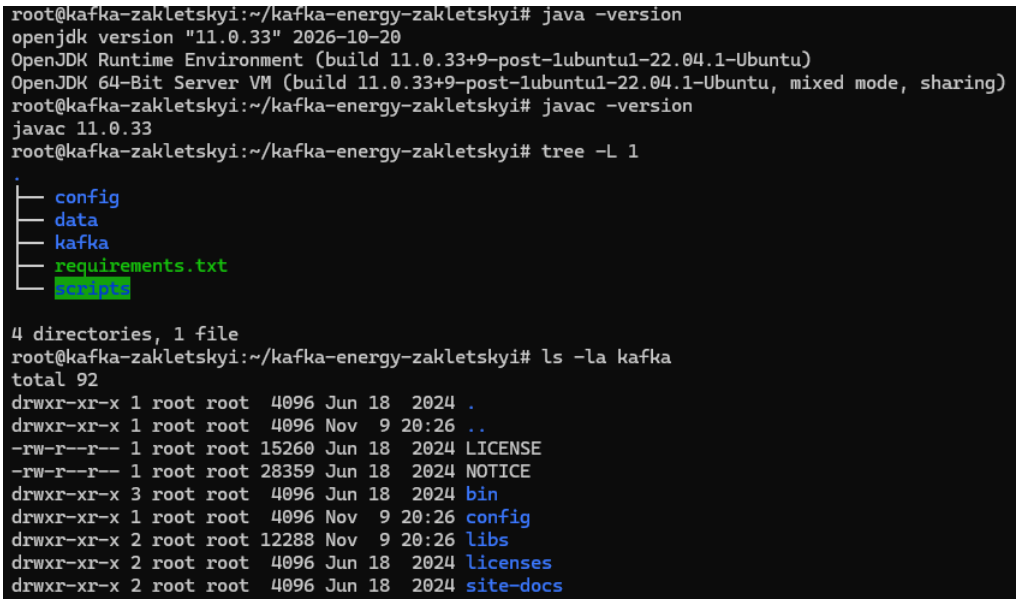


Рисунок 1 – Java, структура проєкту kafka-energy-zakletskyi і вміст Kafka 3.7.1

Крок 3. Запуск Kafka

3.1 Запуск Zookeeper



Рисунок 2 – Запуск ZooKeeper

3.2 Запуск Kafka брокера

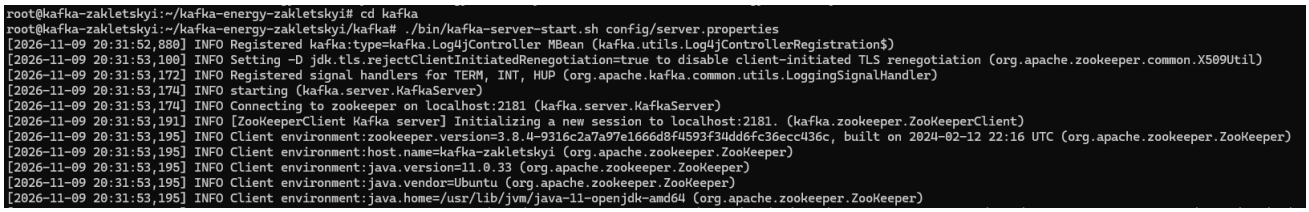


Рисунок 3 – Запуск брокера Kafka 3.7.1

3.3 Перевірка запуску

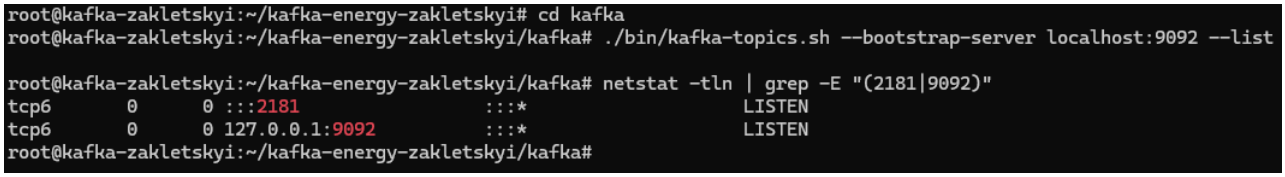


Рисунок 4 – Перевірка запуску: порти 2181 і 9092

Крок 4. Робота з Topic

4.1 Створення Topic для енергетичних даних



Рисунок 5 – Створення топіка power-station-data-zakletskyi

4.2 Операції з Topics в Kafka

```
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic power-telemetry-zakletskyi --partitions 3 --replication-factor 1
Created topic power-telemetry-zakletskyi.
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic grid-alerts-zakletskyi --partitions 2 --replication-factor 1
Created topic grid-alerts-zakletskyi.
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic energy-consumption-zakletskyi --partitions 4 --replication-factor 1
Created topic energy-consumption-zakletskyi.
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic weather-data-zakletskyi --partitions 2 --replication-factor 1
Created topic weather-data-zakletskyi.
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka#
```

Рисунок 6 – Створення додаткових топіків

```
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
energy-consumption-zakletskyi
grid-alerts-zakletskyi
power-station-data-zakletskyi
power-telemetry-zakletskyi
weather-data-zakletskyi
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --describ
e --topic power-station-data-zakletskyi
Topic: power-station-data-zakletskyi    TopicId: YOj40ANERHiknC29nBtsQg PartitionCount: 3       ReplicationFactor: 1   C
onfigs: compression.type=producer,min.insync.replicas=1,cleanup.policy=delete,segment.bytes=1073741824,message.timestamp
.type=CreateTime
        Topic: power-station-data-zakletskyi    Partition: 0    Leader: 0       Replicas: 0     Isr: 0
        Topic: power-station-data-zakletskyi    Partition: 1    Leader: 0       Replicas: 0     Isr: 0
        Topic: power-station-data-zakletskyi    Partition: 2    Leader: 0       Replicas: 0     Isr: 0
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-configs.sh --bootstrap-server localhost:9092 --entity
-type topics --entity-name power-station-data-zakletskyi --describe
Dynamic configs for topic power-station-data-zakletskyi are:
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka#
```

Рисунок 7 – Список топіків, опис і конфігурація топіка power-station-data-zakletskyi

```
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--topic NEW2025-station-data-zakletskyi
Created topic NEW2025-station-data-zakletskyi.
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --delete
--topic NEW2025-station-data-zakletskyi
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka# ./bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
energy-consumption-zakletskyi
grid-alerts-zakletskyi
power-station-data-zakletskyi
power-telemetry-zakletskyi
weather-data-zakletskyi
root@kafka-zakletskyi:~/kafka-energy-zakletskyi/kafka#
```

Рисунок 8 – Видалення топіка

4.3 Перевірка версії та можливостей Kafka 3.7.1



Рисунок 9 – Версії API брокера і лог-директорії

Крок 5. Відправка даних (Producer)

5.1 Тестуємо консольний Producer



Рисунок 10 – Консольний продюсер

5.2–5.3 Створення та запуск Python Producer



Рисунок 11 – Python-продюсер

Крок 6. Отримання даних (Consumer)

6.1 Тестуємо консольний Consumer



Рисунок 12 – Консольний консюмер: JSON від Python-продюсера і повідомлення консольного продюсера

6.2–6.3 Створення та запуск Python Consumer

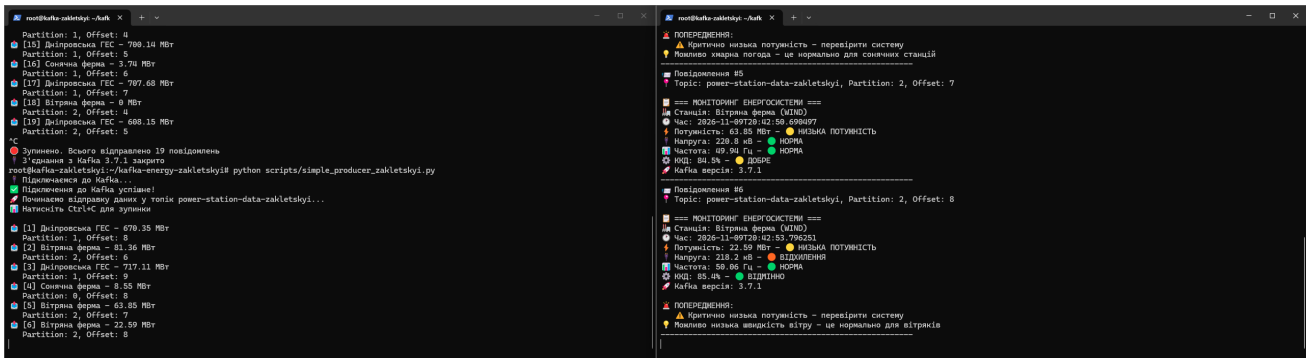


Рисунок 13 – Python-продюсер і Python-консюмер

Крок 7. Моніторинг та адміністрування Kafka

7.1 Перевірка статусу системи

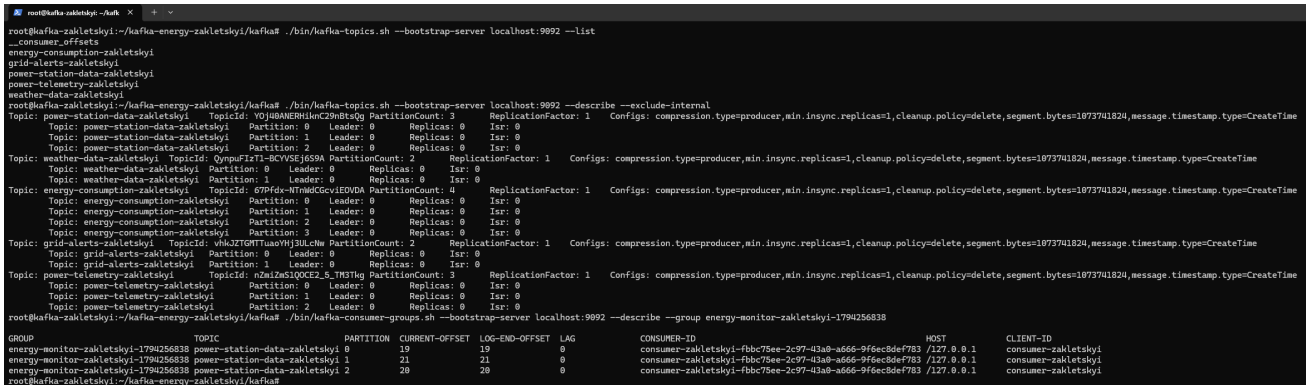


Рисунок 14 – Топіки і група консюмерів energy-monitor-zakletskyi-1794256838

7.2 Перевірка продуктивності Kafka

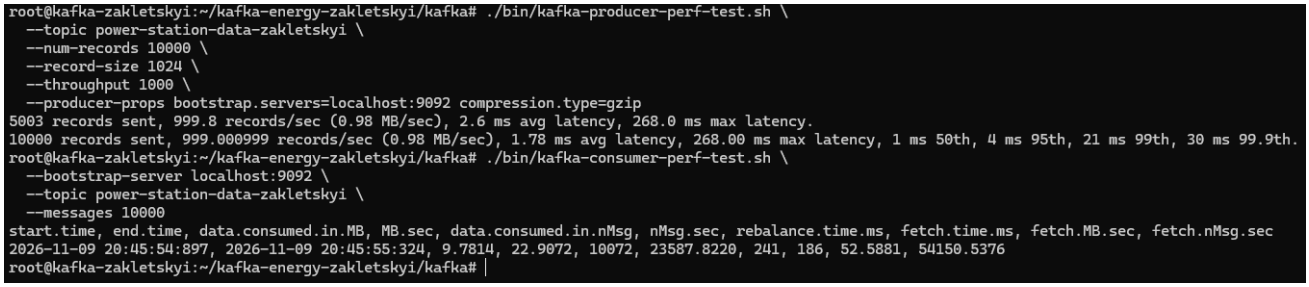


Рисунок 15 – Тести продуктивності продюсера і консюмера

7.3 Логи та налагодження



Рисунок 16 – Лог брокера, конфігурації брокера і топіка, лог-директорії

Крок 8. Правильна зупинка системи

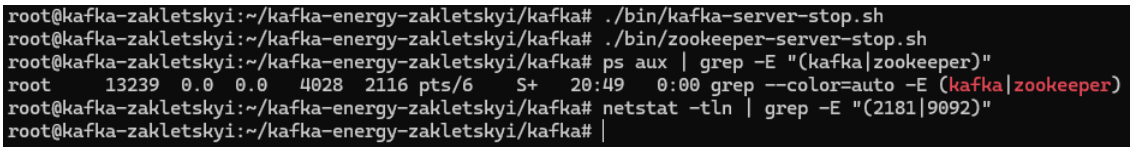


Рисунок 17 – Зупинка Kafka і ZooKeeper, перевірка процесів і портів

Відповіді на контрольні питання

1. Яка різниця між partition та topic в Kafka?

Топік — логічна іменована категорія повідомлень (наприклад, power-station-data-zakletskyi). Партиція — фізична частина топіка: впорядкований журнал, у який повідомлення лише дописуються, і кожне отримує свій offset. Топік складається з однієї або кількох партицій. Вони можуть зберігатися на різних брокерах і читатися паралельно різними консюмерами групи. Порядок гарантується лише в межах партиції, тому повідомлення однієї станції варто надсилати з однаковим ключем: тоді вони потрапляють в одну партицію.

2. Як працює replication в Kafka кластері?

Кожна партиція має replication-factor копій на різних брокерах. Одна копія — лідер: вона приймає запис і віддає дані. Інші (followers) копіюють дані з лідера. Репліки, що не відстають від лідера, утворюють ISR (in-sync replicas). Продюсер з acks=all отримує підтвердження, коли запис є в усіх ISR, а min.insync.replicas задає мінімальну кількість таких реплік, без якої запис не приймається. Якщо брокер-

лідер падає, контролер (через ZooKeeper або KRaft) обирає нового лідера з ISR, і підтверджені дані не втрачаються. У цій роботі брокер один, тому replication factor = 1.

3. Чому Kafka ідеально підходить для IoT-систем?

Kafka добре підходить для IoT-систем, тому що вона:

- приймає мільйони повідомлень за секунду від тисяч датчиків і масштабується додаванням партицій і брокерів;
- розв’язує джерела і споживачів: один потік телеметрії одночасно читають база даних, система тривог і аналітика, кожна у своєму темпі;
- зберігає дані на диску протягом retention: після збою споживач дочитує пропущене, історію можна перечитати;
- зберігає порядок у партиції (ключ — ідентифікатор датчика чи станції) і забезпечує відмовостійкість через реплікацію;
- має екосистему для інтеграції та обробки: Kafka Connect (бази даних, MQTT) і Kafka Streams.

4. Як налаштувати offset management для consumer groups?

Offset-и груп зберігаються в службовому топіку __consumer_offsets окремо для кожної пари «група — партиція». Керувати ними можна так:

- автоматичний коміт: enable.auto.commit=true і auto.commit.interval.ms (у Python-консюмері цієї роботи — кожні 5 с);
- ручний коміт: enable.auto.commit=false і commit() / commitSync() / commitAsync() після обробки повідомлення — це гарантія at-least-once;
- auto.offset.reset (earliest або latest) визначає, звідки читати, якщо збереженого offset-у ще немає;
- з консолі: kafka-consumer-groups.sh --describe показує CURRENT-OFFSET, LOG-END-OFFSET і LAG, а --reset-offsets (--to-earliest, --to-latest, --to-datetime, --shift-by N) з --execute змінює offset-и неактивної групи; у коді позицію змінює seek().

5. Які переваги має schema registry для енергетичних систем?

Schema Registry централізовано зберігає версіоновані схеми повідомлень (Avro, Protobuf, JSON Schema), що дає такі переваги:

– продюсер не відправить повідомлення, що не відповідає схемі (наприклад, power_output_mw рядком замість числа), тож в аналітику і тривоги потрапляє менше некоректних даних;
– при зміні формату перевіряється сумісність: нове поле (наприклад, reactive_power_mvar) не ламає старих консюмерів;
– компактні бінарні повідомлення (Avro) замість JSON дають менший трафік від тисяч датчиків;
– схема стає задокументованим контрактом між системами (SCADA, аналітика, білінг) і різними поколіннями обладнання.

Посилання на лістинг

https://github.com/Frodlik/kafka-energy-zakletskyi